

CHIPONE

集创北方

ICND1069 编程指导

版本说明

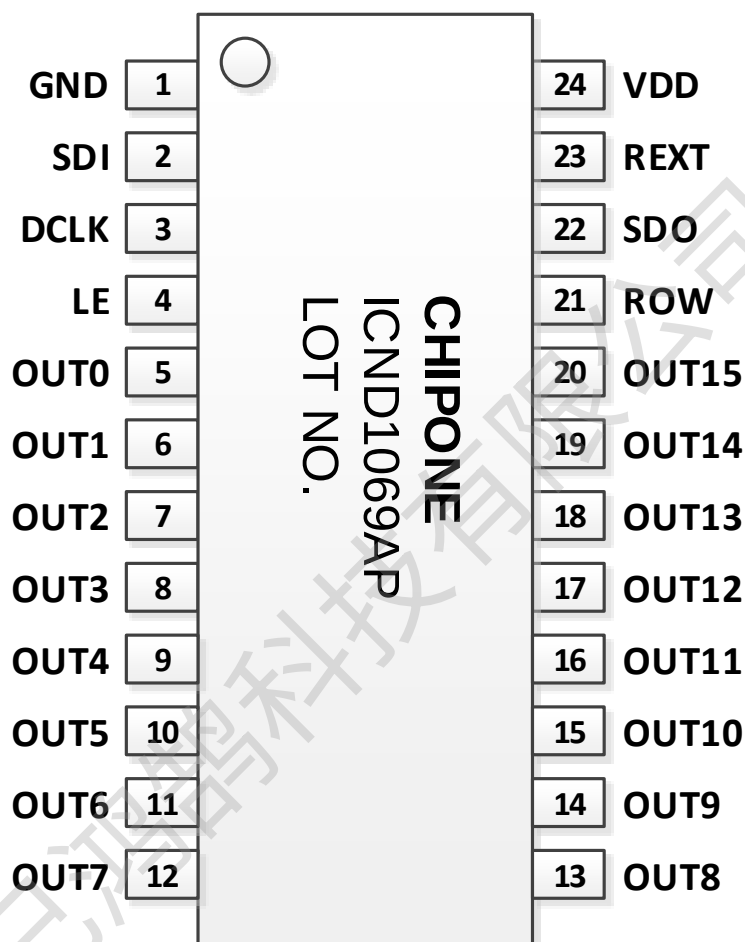
Rev	Date	Description
1.0	2023/01	Initial release
1.1	2024/04	添加开路检测回传内容
1.2	2024/05	更新设置行长度的说明

目录

ICND1069 管脚定义	3
基本指令格式	4
指令定义与配置寄存器	5
控制指令	6
显示驱动方式	8
换行时序	9
显示时序	10
软件主界面说明	11
配置界面	16
开路检测	19
开路数据回读	21
Gamma	23
DLL 使用说明	23
寄存器	24

ICND1069 管脚定义

ICND1069 管脚定义同通用 LED 显示驱动芯片略有不同，采用单 CLK 结构。用 ROW 换行信号取代通常的 GCLK 信号。



管脚编号	管脚名称	功能说明
1	GND	接地端
2	SDI	串行数据输入端
3	DCLK	数据时钟和显示时钟信号，用于发送数据、指令以及 PWM 显示；正常工作时，DCLK 一直存在，不能中断
4	LE	据与指令的锁存端，不同的 LE 长度代表不同指令
5~20	OUT0~OUT15	恒流电流输出端
21	ROW	换行信号，ROW 信号由 0 变为 1 表示一次换行
22	SDO	串行数据输出端
23	REXT	外挂电阻输入端，可调节输出端恒流值
24	VDD	电源输入端

基本指令格式

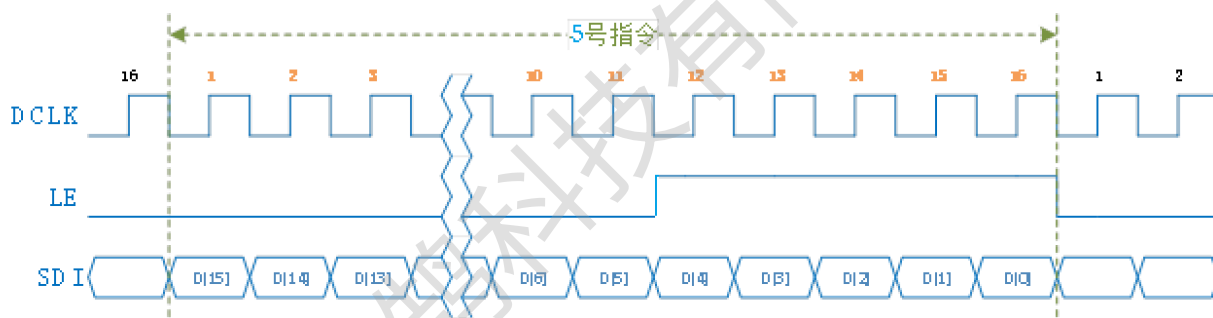
在 ICND1069 系列芯片中，所有的操作通过指令来完成。通常一条指令包括指令编号与指令数据（部分指令不需要指令数据）两部分，当前 ICND1069 共有 8 条指令，指令数据为 16bit 位宽的数据（如果多颗 ICND1069 芯片级联，则数据长度应是 16bit 乘以级联级数）。

ICND1069 中有读、写两类指令，通常读指令用于 Debug，在实际应用中不会使用，因此本文内容主要介绍写指令。

写指令的发出需要 DCLK、LE、SDI 三个端口的配合，其中：

- 1) DCLK 是时钟信号，芯片在 DCLK 上升沿采样 LE 和 SDI 上的数据。
- 2) LE 标识指令编号，具体编号数值由 DCLK 上升沿采样到的 LE 个数决定，即从 DCLK 角度观看到的 LE 长度。LE 的下降沿标志着当前指令结束，该指令立即在芯片内部生效。
- 3) SDI 标识指令数据，在 DCLK 上升沿采样数据。芯片内部有一个 16bit 移位寄存器存储采样到的数据，当 LE 下降沿到来时，该移位寄存器将当前存储的 16bit 数据作为指令数据使用。

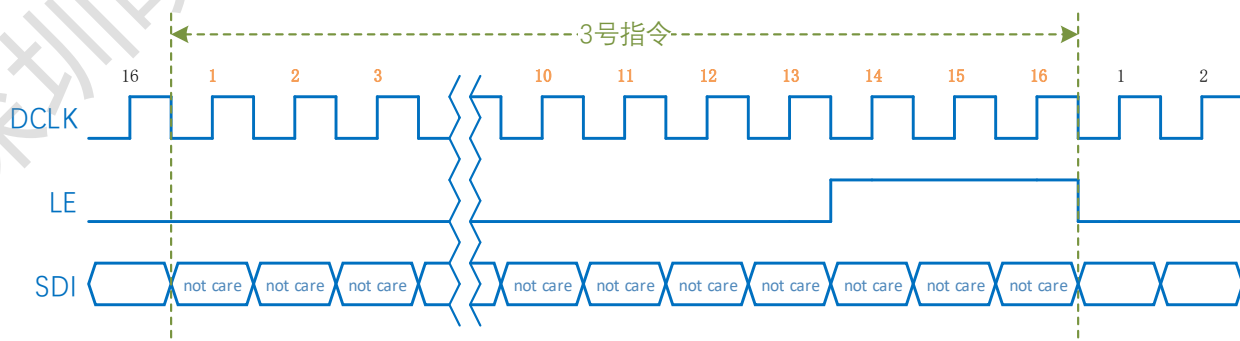
如下图所示，两条绿色虚线中间的部分表示 5 号指令。



5 号指令的格式

5 号指令需要 LE 跨越 5 个 DCLK 上升沿（第 12~16 个 DCLK），也就是 DCLK 上升沿对 LE 的采样数值为 5；SDI 在 16 个周期中总过被 DCLK 上升沿采样 16 次，形成 16bit 数据 D[15:0]，作为 5 号指令的指令数据。在 LE 的下降沿到来时，芯片内部识别到上述指令编号与指令数据，同时使该指令生效。

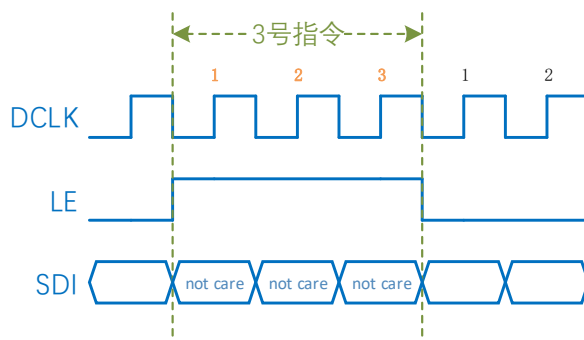
如下图所示，两条绿色虚线中间部分表示 3 号指令。



3 号指令的格式

3 号指令不需要指令数据，所以在发送 3 号指令的过程中的 SDI 数据可以是任意值。

下图是精简的 3 号指令格式，因为 3 号指令不需要指令数据，从节约时间的角度，无需发满 16 个 DCLK，只要 DCLK 个数满足 LE 的长度需求即可。精简的 3 号指令格式



在发送需要数据的指令时，有以下几点需要注意：

- 1) 不能因为上一指令的数据部分与当前指令数据相同，而少发送 DCLK，所以发送数据不能少于 16 个 DCLK。对于多芯片级联，最少 DCLK 个数应乘以级联级数。
- 2) 在 LE 下降沿到来时，如果前面发送的 DCLK 多于 16 个，即 SDI 上的数据多于 16bit，则在时间上最靠近 LE 下降沿的 16bit 数据会作为有效数据使用。
- 3) 指令数据应先发送高位（bit 15），最后发送低位（bit 0）

指令定义与配置寄存器

芯片包含一个简单的 16bit 移位寄存器，灰度值和配置值都会锁存到移位寄存器里面。通过计数 LE 信号的长度来解析控制命令，不同的 LE 长度表示不同的命令。例如长度为 1 的 LE 信号表示“Data Latch”命令，用来控制移位寄存器锁存灰度值，将移位寄存器里的 16bit 数据送给 SRAM。下表列出了所有命令及其释义。

指令定义

指令名称	LE	指令描述
DATA_LATCH	1	锁存 16bit 数据送给 SRAM
VSYNC	3	更新显示数据
WR_CFG	5	写寄存器
RD_CFG	7	读寄存器
SOFT_RST	9	Reset 除寄存器之外的其他功能
EN_OP	11	使能所有输出通道
DIS_OP	12	关闭所有输出通道
PRE_ACT	14	写使能

备注 1： LE 的长度是指当 LE 为高电平时，DCLK 的上升沿个数。如图 1 所示，第一个 LE 信号的长度为 1，亦即该命令为“Data Latch”命令。

备注 2： 执行 PRE_ACT 之后，才可以执行一条或多条 WR_CFG 或 RD_CFG 指令；

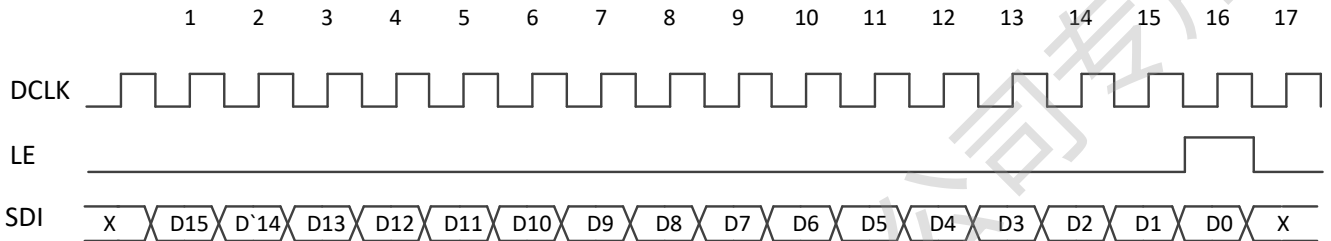
备注 3： 一旦接收到非 WR_CFG 且 RD_CFG 指令，则关闭 PRE_ACT，即不能继续接收 WR_CFG 或 RD_CFG 指令，除非再次发送 PRE_ACT。

备注 4： 发送 DIS_OP 将关闭所有 PWM 输出，直到发送 EN_OP 指令（等待 8 帧）；其他指令的执行没有限制。

控制指令

DATA LATCH 命令

DATA_LATCH 指令用来锁存 16bit 灰度值，其时序如图所示。当检测到 LE 为 1 个 DCLK 宽度时，芯片锁存灰度值，锁存的灰度值为{D15, D14, D13, D12, D11, D10, D9, D8, D7, D6, D5, D4, D3, D2, D1, D0}；否则不锁存。



DATA LATCH 命令时序

DATA_LATCH 指令的功能是将灰度值写入 SRAM。通常一颗 ICND1069 控制 M 颗 LED 灯珠，就要在一帧中执行 M 次 DATA_LATCH 指令。

因为一颗 ICND1069 包含 16 路 PWM 输出通道 (OUT0~OUT15)，为节约成本，通常一颗 ICND1069 会控制 16×N 的 LED 阵列（在实际中，N 一般为 8、16、32 等，可配置），即一条 OUT 通道连接 N 颗 LED，在显示时采用 N 行轮流显示的方式工作，N 行的轮选逻辑由片外电路实现。

例如，某颗 ICND1069 控制 16×8 的阵列，则在一帧中需要发送 128 次 DATA_LATCH 指令，为 128 颗 LED 写入灰度数据。第一次 DATA_LATCH 写入的数据对应于第一行（即外部轮选逻辑在一帧中首先选中的一行 LED）OUT15 连接的 LED，第二次 DATA_LATCH 对应于第一行 OUT14 连接的 LED，以此类推，直至写完所有的 8 行 128 个 LED。

例如，ICND1069 在 64 扫模式下，DATA_LATCH 锁存数据顺序：

Data Order	Line	Channel
1	Line 1	Channel 15 (OUT15)
2		Channel 14 (OUT14)
.....		
16		Channel 0 (OUT0)
17	Line 2	Channel 15 (OUT15)
18		Channel 14 (OUT14)
.....		
32		Channel 0 (OUT0)
.....		
1009	Line 64	Channel 15 (OUT15)
1010		Channel 14 (OUT14)
.....		
1024		Channel 0 (OUT0)

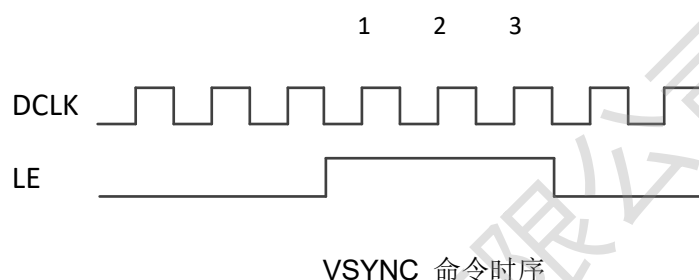
VSYNC 指令

VSYNC 是帧同步指令。当完成一帧所有数据写入与显示时，需要发送 VSYNC 指令，控制 ICND1069 进入到下一帧的工作中。

每一帧的图像数据均以 LE 表示的 VSYNC 为起点，即当检测到 LE 的高电平为 3 个 DCLK 宽度时，后续接收的第一个数据表示该帧的第一个有效图像数据，且按照上节中的顺序传输数据。

需要说明的是：在发送数据期间，可以插入其它的配置命令。

VSYNC 正常情况下每帧发送一次。



PRE_ACT 指令

PRE_ACT 指令用于暂时关闭芯片写保护功能。执行 PRE_ACT 之后，才可以执行一条或多条 WR_CFG 或 RD_CFG 指令。

一旦接收到非 WR_CFG 且 RD_CFG 指令，则关闭 PRE_ACT，即不能继续接收 WR_CFG 或 RD_CFG 指令，除非再次发送 PRE_ACT。

WR_CFG 命令

WR_CFG 命令用来写芯片的寄存器。写寄存器的过程是：先执行 PRE_ACT 命令，并配置寄存器 [0x00]=8'hAA, [0x01]=8'hAA 再执行一个或多个 WR_CFG 命令；结束时，需要配置寄存器 [0x00]=8'h55, [0x01]=8'h55;

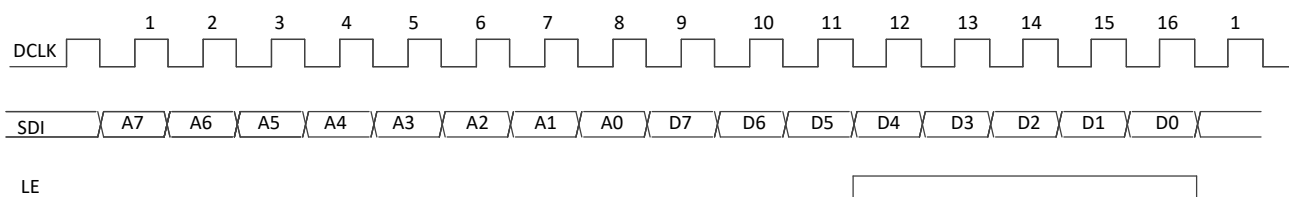
写寄存器过程中不能出现 DATA LATCH 命令执行，WR_CFG 时，LE 的宽度为 5 个 DCLK，其中，先输入的 8bit 为寄存器地址选择，后输入的 8bit 为所选择的寄存值。

如下图：假定

{A7, A6, A5, A4, A3, A2, A1, A0} = 8'b1011_0101;

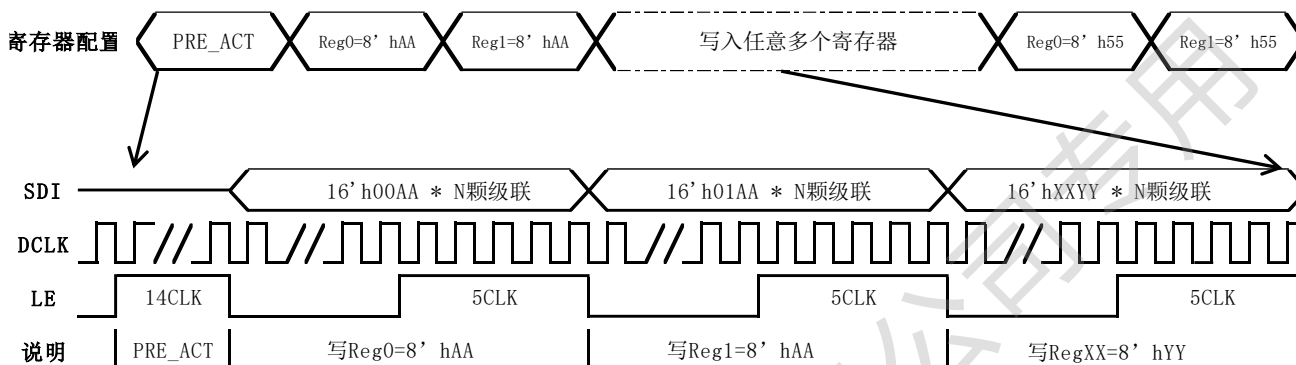
{D7, D6, D5, D4, D3, D2, D1, D0} = 8'b1100_1001;

即表示将寄存器 0xb5 (8'b1011_0101) 设置为 8'b1100_1001。



写寄存器时序步骤:

- 1 发送 PRE_ACTIVE 指令 (LE 宽度为 14)
- 2 配置寄存器[0x00]=8'hAA, [0x01]=8'hAA
- 3 配置其它寄存器 (一个或多个)
- 4 配置寄存器[0x00]=8'h55, [0x01]=8'h55;



EN_OP 与 DIS_OP 指令

EN_OP 与 DIS_OP 分别用于开启通道 (OUT0~OUT15) 输出和关闭通道输出。这两条指令用于显示屏开启时或显示过程中更换寄存器配置时, 避免显示器花屏显示不可预期的图像。

芯片上电之后, 通道状态默认关闭, 在完成第一帧数据接收后, 要先发送 EN_OP 指令开启通道输出。

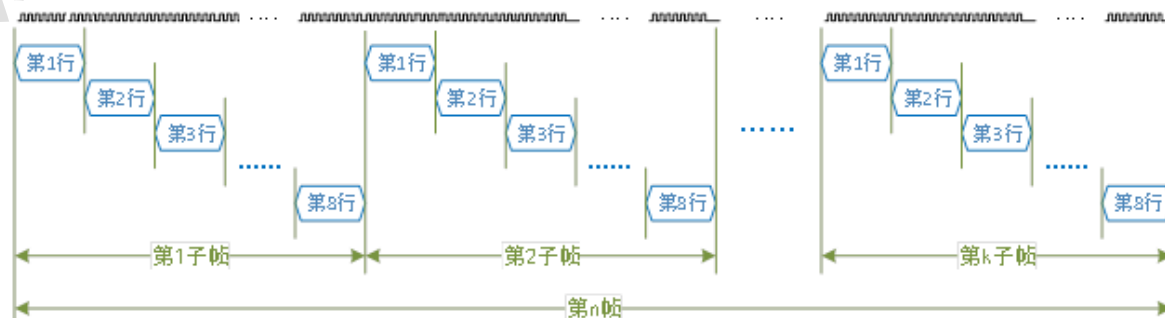
通道状态完全受这两条指令控制, 没有自开启或自关闭的功能, 即 EN_OP 发送后, 通道状态一直维持开启, 直至收到 DIS_OP 指令; 同理 DIS_OP 发送后, 通道状态一直维持关闭, 直至收到 EN_OP 指令 (等待 8 帧)。

显示驱动方式

ICND1069 锁存数据与普通 PWM 芯片一样依赖于 DCLK, 而 GCLK 则不需要外部提供, GCLK 可以通过采集 DCLK 等信号由内部生成。每行的显示时间采用 DCLK 计数。

ICND1069 属于高刷新 LED 驱动芯片, 如芯片的高刷新率通过将一个显示帧划分为若干个显示子帧来实现, 每个子帧中, 所有行的 LED 都会有一段显示时间。对于 1 颗 LED, 它将所有子帧的显示时间相加, 构成一帧完整的灰阶。

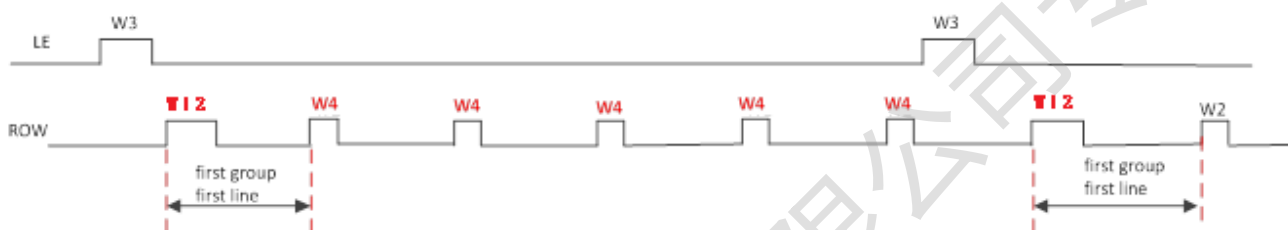
下图为一个显示帧的示例



换行时序

ICND1069 采用 ROW 来表示一行显示的开始（与行控制信号同步），高电平宽度为 12 个或 4 个 DCLK 的宽度。当显示第一组（group）的第一行数据时，ROW 的高电平宽度为 12 个 DCLK 宽度，其它情况为 4 个 DCLK 宽度。

每设置的分组数*扫描行数个换行 ROW 信号，发送一次第 1 组第 1 行的高电平宽度（12 个 DCLK，请注意必须与显示模组上的行信号同步上：与第 1 行对应），不用关心此时的显示数据处于哪一具体分组。



如上图, W3 指高电平宽度为 3 个 DCLK, W12 指高电平为 12 个 DCLK, W4 表示高电平为 4 个 DCLK。

当 ROW 的高电平为 12 个 DCLK 时，表示要进行第一个分组（group）的第一行的显示，那么下一个 W4，就是第一组的第二行的显示。

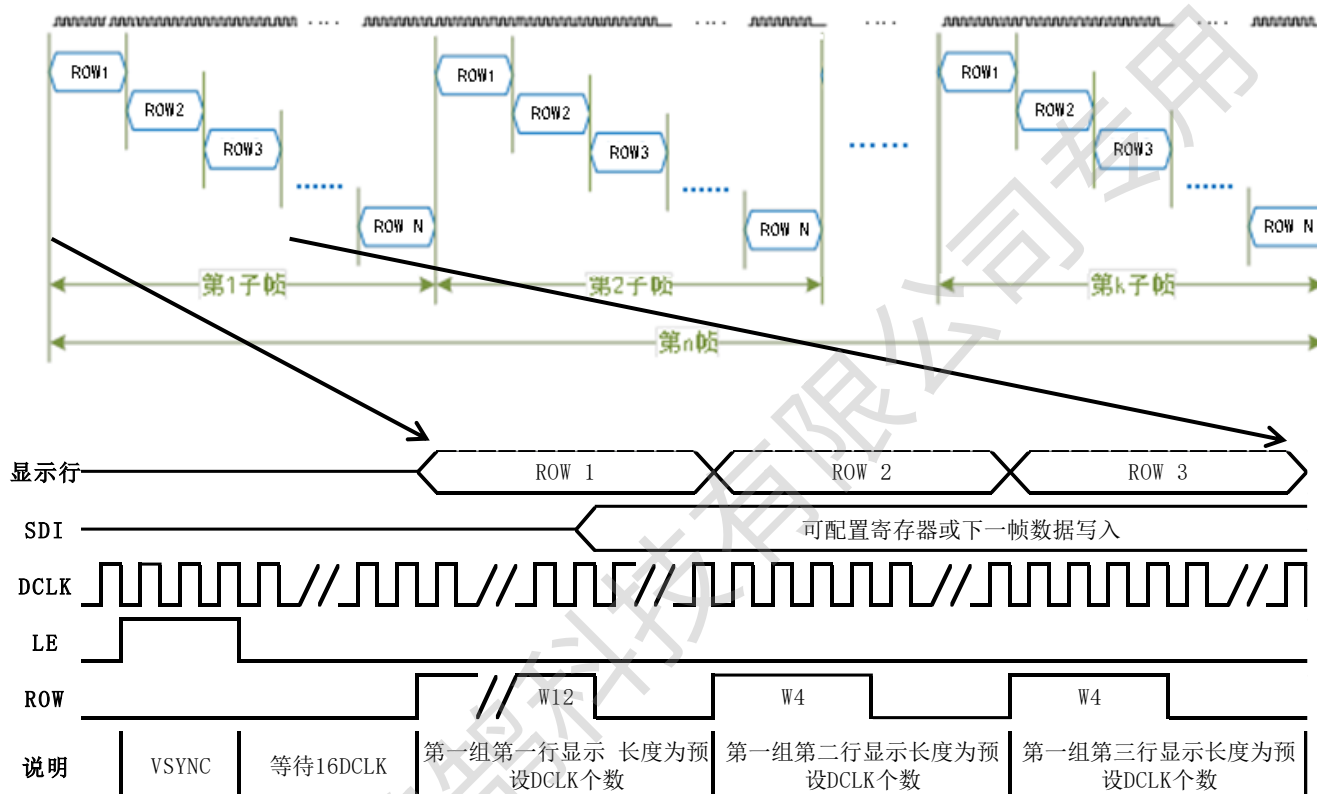
每行显示时间和换行时间的划分由内部寄存器配置决定。

备注：

在接收卡内，换行的时序控制模块应采用 DCLK 做为参考时钟，每过 N 个 DCLK(可寄存器配置)就换行，并产生 ROW 信号。

显示时序

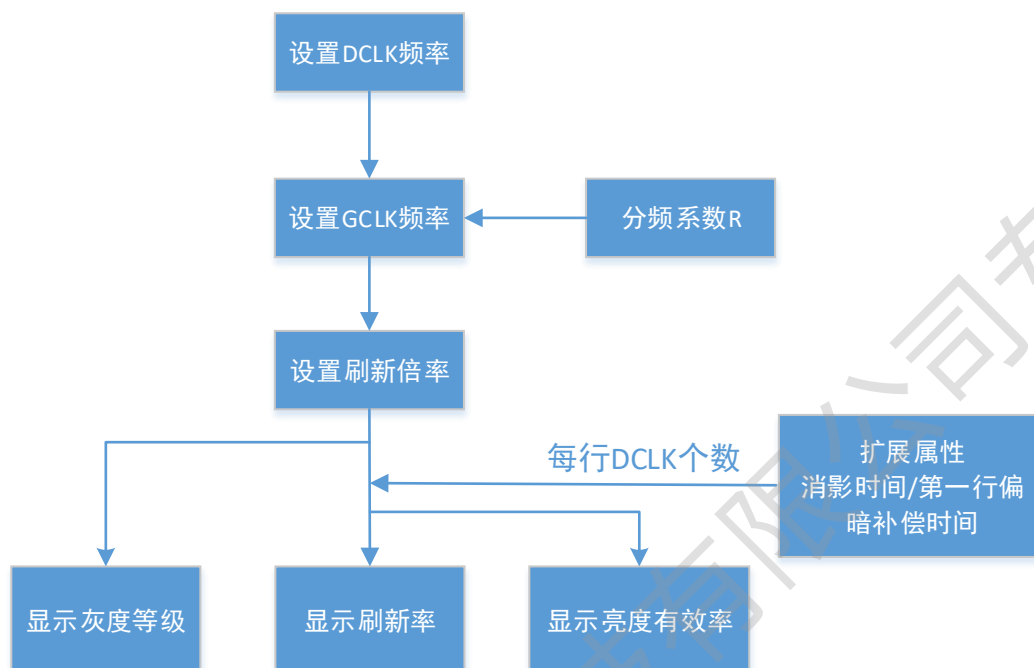
- 1 帧启示发送 VSYNC 之后等待 16 个 DCLK，发送 ROW12 信号，显示第 1 行。
- 2 等待该行 DCLK 个数达到配置值时，发送 ROW4 信号，进行换行。
- 3 重复步骤 2，直到数据组数全部打完，等待下一帧开始。



备注：

- 1 DCLK 个数通过界面进行配置；
- 2 配置数据组数，reg0x03[6:0]=刷新率/帧率-1；
- 3 显示过程中，可同时进行寄存器配置及数据锁存，与显示时序无关；
- 4 每帧保证为整数个(第 1 行至第 n 行)显示循环。

软件主界面说明



1. DCLK 频率

DCLK 频率为控制器提供。(建议默认 12.5MHz, 常规应用 $4\text{MHz} < \text{DCLK} < 16\text{MHz}$)

2. 设置 GCLK 分频系数

方式 1): 根据当前设置的 DCLK 频率, 不同的分频系数 R, 自动计算出 GCLK 频率形成下拉框;

方式 2): 根据当前设置的 DCLK 频率, 设置不同的分频系数 R, 显示出当前的 GCLK 频率。

设置 GCLK 分频系数 R

$$R = \frac{P}{M \times N}$$

$P = \text{reg0x05}[4:0]$ (R,G,B 寄存器同步) PLL_LOOP_DIV

00000/00001: 1 分频

其它: DIV<4:0>分频

$M = \text{reg0x04}[4:0]$ (R,G,B 寄存器同步) PLL_PRE_DIV

00000/00001: 1 分频

其它: DIVI<4:0>分频

$N = \text{reg0x06}[1:0]$ (R,G,B 寄存器同步) PLL_POST_DIV

00/01: 1 分频

10: 2 分频

11: 4 分频

PLL 相关配置以及要求如下

界面上添加选项:

不勾选:选择第一种表格配置(和 2065 相同)

勾选选项:选择第二种表格配置(优化分频比)

第一种分频表格

$1.4\text{MHz} < \text{FDCLK}/\text{PLL_PRE_DIV} < 10\text{MHz}$ (即 ICND1069 最低 DCLK 频率需大于 4MHz)

$2.25\text{MHz} < \text{FDCLK} * \text{PLL_LOOP_DIV} / \text{PLL_PRE_DIV} < 100\text{MHz}$

$3. \text{FGCLK} = \text{FDCLK} * \text{PLL_LOOP_DIV} / (\text{PLL_PRE_DIV} * \text{PLL_POST_DIV})$

当 $4\text{MHz} < \text{DCLK} < 8\text{MHz}$ 时:

分频系数 R 可设定的值及对应 P,M,N 值如下表, 最低 GCLK 频率可设置为 7MHz, 最大可设置为 80MHz

R	P	M	N
1.75	7	1	4
2	8	1	4
2.5	10	1	4
3	12	1	4
3.5	7	1	2
4	8	1	2
4.5	9	1	2
5	10	1	2
5.5	11	1	2
6	12	1	2
7	7	1	1
8	8	1	1
9	9	1	1
10	10	1	1

当 $8\text{MHz} < \text{DCLK} < 16\text{MHz}$ 时:

分频系数 R 可设定的值及对应 P,M,N 值如下表, 最低 GCLK 频率可设置为 7MHz, 最大可设置为 96MHz

R	P	M	N
0.875	7	2	4
1	8	2	4
1.25	10	2	4
1.5	12	2	4
1.75	7	2	2
2	8	2	2
2.5	10	2	2
3	12	2	2
3.5	7	2	1
4	8	2	1
4.5	9	2	1
5	10	2	1

5.5	11	2	1
6	12	2	1

当 DCLK 在这两种不同情况下分频系数 R 为两个不同下拉框。

第二种分频表格

1. $1\text{MHz} < \text{FDCLK}/\text{PLL_PRE_DIV} < 10\text{MHz}$

2. $8\text{MHz} < \text{FDCLK} * \text{PLL_LOOP_DIV} / \text{PLL_PRE_DIV} < 100\text{MHz}$

3. $\text{FGCLK} = \text{FDCLK} * \text{PLL_LOOP_DIV} / (\text{PLL_PRE_DIV} * \text{PLL_POST_DIV})$

当 $4\text{MHz} < \text{DCLK} < 8\text{MHz}$ 时:

分频系数 R 可设定的值及对应 P,M,N 值如下表, 最低 GCLK 频率可设置为 7MHz, 最大可设置为 80MHz

R	P	M	N
1.75	7	4	1
2	4	2	1
2.5	5	2	1
3	6	2	1
3.5	7	2	1
4	8	2	1
4.5	9	2	1
5	10	2	1
5.5	11	2	1
6	12	2	1
7	7	1	1
8	8	1	1
9	9	1	1
10	10	1	1

当 $8\text{MHz} < \text{DCLK} < 16\text{MHz}$ 时:

分频系数 R 可设定的值及对应 P,M,N 值如下表, 最低 GCLK 频率可设置为 7MHz, 最大可设置为 96MHz

R	P	M	N
0.875	7	8	1
1	4	4	1
1.25	5	4	1
1.5	6	4	1
1.75	7	4	1
2	4	2	1
2.5	5	2	1
3	6	2	1
3.5	7	2	1
4	8	2	1
4.5	9	2	1
5	10	2	1
5.5	11	2	1



当 DCLK 在这两种不同情况下分频系数 R 为两个不同下拉框。

3. 设置刷新倍率

可设可设置为 1, 2, 4, 8, 16, 32 和手动设置。

对应寄存器设置关系如下，即主界面设置为相应刷新倍率的时候寄存器设置值，默认刷新倍率 8。

刷新倍率 1: reg07[7:0]=255 (R,G,B 寄存器同步)

刷新倍率 2: reg07[7:0]=128

刷新倍率 4: reg07[7:0]=64

刷新倍率 8: reg07[7:0]=32

刷新倍率 16: reg07[7:0]=16

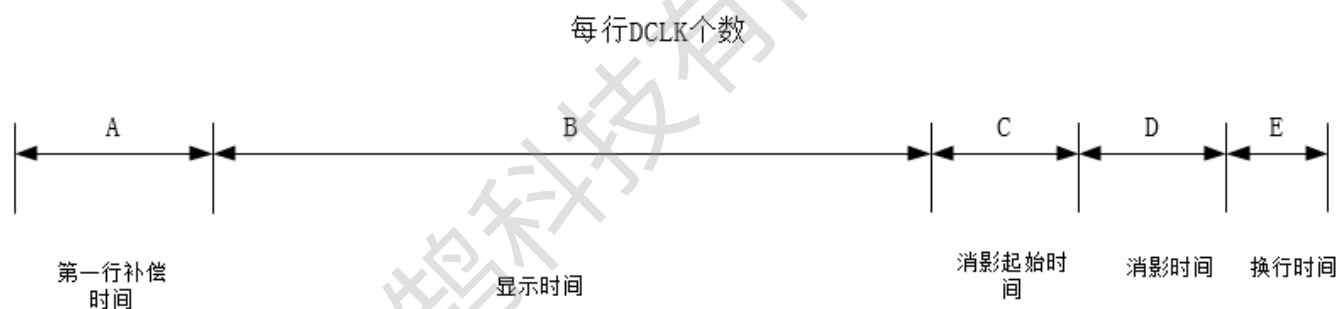
刷新倍率 32: reg07[7:0]=8

手动设置: reg07[7:0]可设置为 1~255 任意值，弹出帧间隔时间（即原 DCLK 余量）显示单位为时间 ms。

4. 设置行长度

每行的 DCLK 个数 N 说明（Row 信号下降沿到下一个 Row 信号下降沿之间）

DCLK 总个数由 A,B,C,D,E 决定。N=A+B+C+D+E



$$N = \text{Fixed} \left(\frac{0x0e[4:0] \times 2 + 0x7[7:0] \times 4 + 0x0f[3:0] \times 4 + 0x0d[4:0] \times 2}{R} + 1 \right) + \text{换行时间}$$

N = 第一行补偿时间 + 显示时间 + 消影起始时间 + 消影时间 + 换行时间

A 第一行补偿时间: $0x0e[4:0] \times 2 / R$

B 显示时间: $0x7[7:0] \times 4 / R + 1$

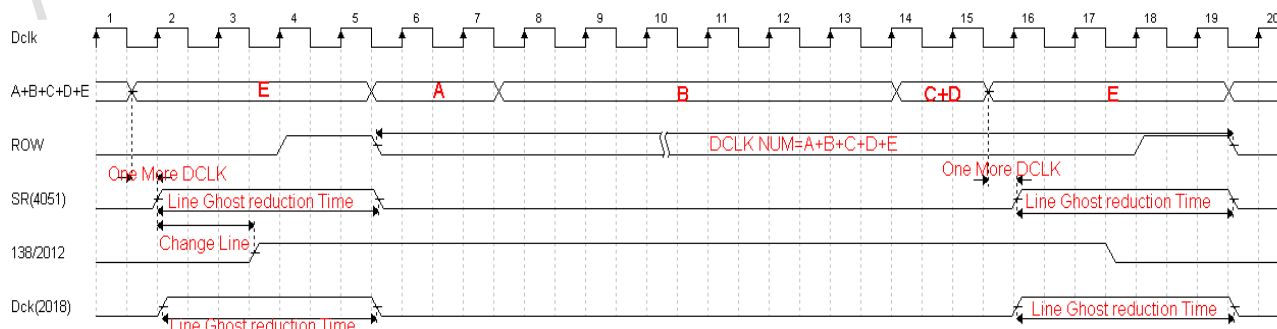
C 消影起始时间: $0x0f[3:0] \times 4 / R$

D 消影时间: $0x0d[4:0] \times 2 / R$

E 换行时间: 最小值为 2

说明: 针对帧率自适应的应用, 建议 $N = A + B + C + D + E > 33$

此处需要分别计算 R,G,B 的每行 DCLK 个数 N1, N2, N3 在取 N1, N2, N3 的最大值作为最后结果 N。



A 的起始位置与 E 的结束位置均为 Row 下降沿,并与行消隐时间结束点对齐
E 时间比行消隐时间大 1 个 DCLK 周期。(行消隐时间可由软件设置)
换行时刻从行消隐时间起点向后可调 (可由软件设置)

说明:

对于控制系统来说, 根据寄存器设置的 ABCD 值和软件设置的行消隐时间即可得出一行的时间。同时确定 ROW 的长度。根据 Row 的下降沿位置, 根据行消隐时间和换行时刻可以得出换行位置。

换行位置到 Row 信号下降沿为 2018 的 CLK 高电平宽度。

对于 2012 来说只需确定换行位置

对于 4051 来说, SR 信号是行消隐时间起始到余辉结束时刻 (比行消隐时间结束点要早), A 信号的沿在换行位置, 被 SR 包住。

5. 显示刷新率

刷新率=分组数($\text{reg0x03}[6:0]+1$) * 帧率

6. 显示灰度等级

灰度等级= ($\text{reg0x03}[6:0]+1$) * ($\text{reg07}[7:0]$) * 4

通过寄存器配置的灰度等级

$G = (\text{reg0x03}[6:0]+1) * (\text{reg07}[7:0]) * 4$

若 $4096 < G \leq 8192$ 主界面需显示 13bit, $8192 < G \leq 16384$ 主界面需显示 14bit, $16384 < G \leq 32768$ 主界面需显示 15bit, $32768 < G \leq 65536$ 主界面需显示 16bit。

7. 显示亮度有效率

亮度有效率= (每行 GCLK 个数 * GCLK 周期) / (每行 DCLK 个数 * DCLK 周期)

每行 GCLK 个数/4= $\text{reg0x07}[7:0]$

GCLK 周期=1/GCLK 频率

DCLK 周期=1/DCLK 频率

配置界面

- 消隐等级 R/G/B 分别可调**
下拉选项: 0-31 共 32 级 默认: R/G/B=31/25/23
Reg0x1E[4:0], 00000 对应 31 级
- 消隐时间 R/G/B 分别可调**
下拉选项: 0-31 共 32 级 默认: R/G/B=2/2/2
Reg0x0D[4:0], 00000 对应 0 级
- 低灰均匀性 R/G/B 分别可调**
下拉选项: 0-4 共 5 级 默认: R/G/B=3
Reg0x10[7:5], 000 对应 0 级, 010 对应 1 级, 100 对应 2 级, 110 对应 3 级, 111 对应 4 级
- 第一行偏暗补偿 R/G/B 分别可调**
下拉选项: 0-31 共 32 级, 默认: R/G/B=8/10/16
Reg0x18[5:4] (高 2bit) + reg0x19[7:5] (低 3bit), 00000(reg0x18[5:4]+reg0x19[7:5])对应 0 级

5. 第一行偏暗补偿步阶 默认不勾选 R/G/B 分别可调
 勾选后：1-3 共 3 级， 默认：R/G/B=1/2/2
 Reg0x19[4:3], 00 对应 1 级 01 对应 2 级 11 对应 3 级

5. 第一行偏暗补偿时间 R/G/B 分别可调
 下拉选项：0-31 共 32 级， 默认：R/G/B=6/6/6
 Reg0x0E[4:0] 00000 对应 0 级

6. 第一行偏暗 增强 R/G/B 分别可调
 勾选项 默认 R/G/B=不勾选
 Reg0x24[3] 1 对应勾选

7. 低灰偏色补偿 R/G/B 分别可调
 下拉选项：0-7 共 8 级 默认 R/G/B=0/0/0
 Reg0x23[3:1] 000 对应 0 级

GCLK 频率	低灰偏色补偿等级
GCLK<25.9MHz	0-7
25.9M≤GCLK<30.3M	0-6
30.3M≤GCLK<36.3M	0-5
36.3M≤GCLK<45.5M	0-4
45.5M≤GCLK<60.6M	0-3
60.6M≤GCLK<90.9M	0-2
90.9M≤GCLK≤96M	0-1

8. 低灰偏色补偿 增强 R/G/B 分别可调
 勾选项 默认 R/G/B=不勾选
 Reg0x0f[4] 1 对应勾选

9. 耦合增强 R/G/B 同步
 勾选项 默认 R/G/B 不勾选
 Reg0x20[6] 1 对应勾选

10. 耦合优化 1 R/G/B 分别可调
 下拉选项：0-255 共 256 级（上限为 reg0x07[7:0]） 默认：R/G/B=31/30/29
 Reg0x0C[7:0], 000000000 对应 0 级, reg0x0c[7:0]≤reg0x07[7:0]

11. 耦合优化 2 启用 R/G/B 分别可调
 勾选项 默认：R/G/B=不勾选/勾选/勾选
 Reg0x1F[5], 勾选为 1

12. 耦合优化 2 增强 R/G/B 同步
 勾选项 默认：不勾选
 Reg0x22[1], 勾选为 1

13. 耦合优化 2

R/G/B 分别可调

下拉选项: 0-31 共 32 级 默认: R/G/B=2/9/13

Reg0x20[4:0], 00000 对应 0 级

14. 慢速开启

R/G/B 分别可调

下拉选项: 1-4 共 4 级 默认: R/G/B=3/3/3

Reg0x1D[7:6] 00 对应 1 级

15. 拐点电压

R/G/B 分别可调

下拉选项: 0-3 共 4 级 默认: R/G/B=1

Reg0x1D[3:2], 00 对应 1

16. 渐变优化 R/G/B 同步

勾选项 默认: 不勾选

Reg0x24[1] 0 对应不勾选 1 对应勾选

17. 输出电流调节 GAIN=50%~200%, 对应寄存器 Reg0x1C[7:0], 默认为 Reg1C[7:0]=C0

Reg0x1C[6:0]≥1000000

 $IGAIN = (\text{Reg0x1C}[7] + 1) * \text{Reg0x1C}[6:0] / 128$

18. 电流验证模式

R/G/B 同步

勾选项 默认: 不勾选

勾选: Reg0x04[7]=1 Reg0x05[7]=1

不勾选: Reg0x04[7]=0 Reg0x05[7]=0

19. 黑屏节能

R/G/B 分别可调

下拉选项: (效果优先, 性能优先, 节能优先) 默认: R/G/B=效果优先

效果优先: reg0x13[4]=0 reg0x13[2]=0 reg0x23[7]=0

性能优先: reg0x13[4]=1 reg0x13[2]=1 reg0x23[7]=0

节能优先: reg0x13[4]=1 reg0x13[2]=1 reg0x23[7]=1

20. 去除坏点

☐ 启用

去除坏点

勾选为启用 (reg0x16[1]=1), 不勾选为关闭此功能 (reg0x16[1]=0)

开路检测以及去除坏点, 默认为上电 3 秒后自动检测, 可手动点击 (去除坏点)

开路检测

增强模式 1

1. 上电 ≥ 60 帧后, 执行开路检测。
2. 发送 Pre_active
3. `Wr reg0x00 = 8'haa; wr reg0x01=8'haa; //set password`
4. 将电流增益调整到界面自定义相对应的值
5. 等待一帧时间, 按照以下步骤执行开路检测
6. `Wr reg0x16 = 8'h01; //enable open detect`
7. 发送 Row, 即先设置 Row 为高, 然后设置 ROW 为低, 高电平时间(建议为 400uS)为检一行所需要的时间, 低电平时间(建议 ≥ 16 个 DCLK)为数据采集和锁存时间。DCLK 连续发送。
8. 换行, 需要扫描多少行, 步骤 7 就执行多少次。
9. 检测完毕后, `Wr reg0x16 = 8'h02`, 电流增益恢复为开路检测前预设值; //关闭 open_detect,同时 enable open。
10. 换行时刻建议与 ROW 的上升沿对齐
11. 开路检测期间, $LE=0$ 。

增强模式 2

1. 上电 ≥ 60 帧后, 执行开路检测。
2. 发送 Pre_active
3. `Wr reg0x00 = 8'haa; wr reg0x01=8'haa; //set password`
4. `Wr reg0x1c = 8'h39 //调整侦测电流增益为 30%`
5. 等待一帧时间, 按照以下步骤执行开路检测
6. `Wr reg0x16 = 8'h01; //enable open detect`
7. 发送 Row, 即先设置 Row 为高, 然后设置 ROW 为低, 高电平时间(建议为 400uS)为检一行所需要的时间, 低电平时间(建议 ≥ 16 个 DCLK)为数据采集和锁存时间。DCLK 连续发送。
8. 换行, 需要扫描多少行, 步骤 7 就执行多少次。
9. 检测完毕后, `Wr reg0x16 = 8'h02`, 电流增益恢复为开路检测前预设值; //关闭 open_detect,同时 enable open。
10. 换行时刻建议与 ROW 的上升沿对齐
11. 开路检测期间, $LE=0$ 。

增强模式 3

1. 上电 ≥ 60 帧后, 执行开路检测。
2. 发送 Pre_active
3. `Wr reg0x00 = 8'haa; wr reg0x01=8'haa; //set password`
4. `Wr reg0x1c = 8'h72 //调整侦测电流增益为 60%`
5. 等待一帧时间, 按照以下步骤执行开路检测
6. `Wr reg0x16 = 8'h01; //enable open detect`
7. 发送 Row, 即先设置 Row 为高, 然后设置 ROW 为低, 高电平时间(建议为 400uS)为检一行所需要的

时间, 低电平时间(建议 ≥ 16 个 DCLK)为数据采集和锁存时间。DCLK 连续发送。

8. 换行, 需要扫描多少行, 步骤 7 就执行多少次。

9. 检测完毕后, $Wr\ reg0x16 = 8'h02$, 电流增益恢复为开路检测前预设值; //关闭 open_detect,同时 enable open。

10. 换行时刻建议与 ROW 的上升沿对齐

11. 开路检测期间, $LE=0$ 。

增强模式 4

1. 上电 ≥ 60 帧后, 执行开路检测。

2. 发送 Pre_active

3. $Wr\ reg0x00 = 8'haa$; $wr\ reg0x01=8'haa$; //set password

4. $Wr\ reg0x1c = 8'hb9$ //调整侦测电流增益为 90%

5. 等待一帧时间, 按照以下步骤执行开路检测

6. $Wr\ reg0x16 = 8'h01$; //enable open detect

7. 发送 Row, 即先设置 Row 为高, 然后设置 ROW 为低, 高电平时间(建议为 400uS)为检一行所需要的时间, 低电平时间(建议 ≥ 16 个 DCLK)为数据采集和锁存时间。DCLK 连续发送。

8. 换行, 需要扫描多少行, 步骤 7 就执行多少次。

9. 检测完毕后, $Wr\ reg0x16 = 8'h02$, 电流增益恢复为开路检测前预设值; //关闭 open_detect,同时 enable open。

10. 换行时刻建议与 ROW 的上升沿对齐

11. 开路检测期间, $LE=0$ 。

十字架消除设置

☐ 启用去除坏点

去除坏点

☒ 去除坏点增强模式使能

R	<	>	100.00	%
G	<	>	100.00	%
B	<	>	100.00	%

☐ 电流增益调整

十字架消除设置

☐ 启用去除坏点

去除坏点

☒ 去除坏点增强模式使能

增强模式1

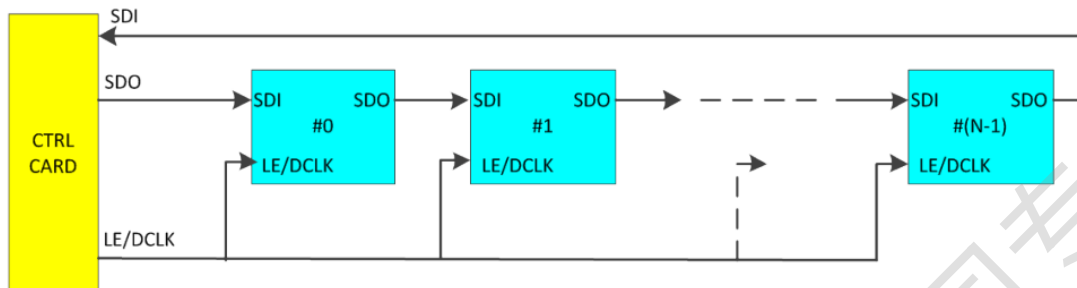
增强模式2

增强模式3

增强模式4

开路数据回读

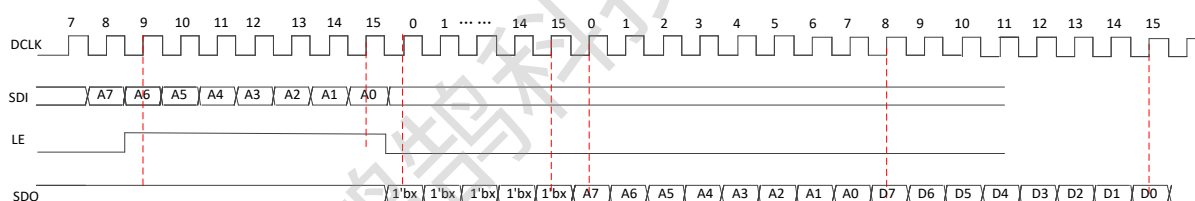
ICND1069 开路数据可以通过 SDO 回读到控制卡。下图为回读开路数据示意图。



开路数据回读通过寄存器读出功能实现。

RD_CFG 命令

RD_CFG 命令用来读出芯片的寄存器。读寄存器的过程是：SDI 将需要读的寄存器地址置于低 8bit 送入，同时 LE 包含 7 个 DCLK 进行锁存，等待 16 个空 DCLK 后，读出 16bit 数据，其中高 8bit 为寄存器地址，低 8bit 为寄存器值。



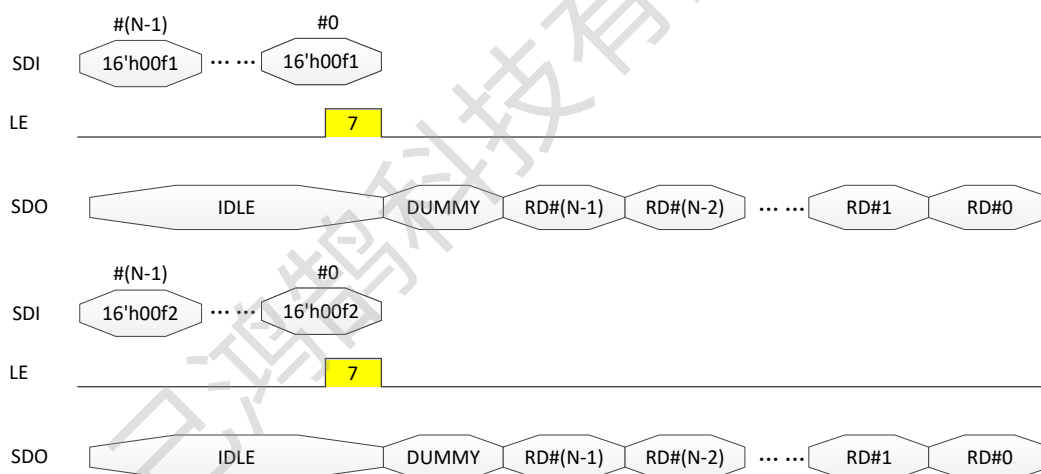
需要读出 N 个芯片数据时，需要一次送入 N 个寄存器地址，通过 LE 同时锁存到芯片。读出时级联末尾的芯片优先读出，依次向前读出芯片寄存器数据。

开路数据回读时序



1. 等待开路检测完成;
2. 发送 Pre_active;
3. WR REG[0x00] = 8'hAA; WR REG[0x01]=8'hAA; \\Set password
4. 设置要读出扫描行（一次读出一行，通过循环设置实现全部读出）;
WR REG[0x72] = 8'h80+M-1; \\M 为需要读出的扫描行，即需要读出第一行时，寄存器写入 0x80
WR REG[0x73] = 8'h62
5. 读出开路数据，因读出数据为 8bit，所以 16 个通道需要分两次读出；
CH[15:8]对应寄存器为 0xF1；
CH[7:0]对应寄存器为 0xF2；
6. 重复步骤 2~5，完成下一行的数据读出。（可以一次读完全部行或每帧读出一行）

下图分别为高 8 通道与低 8 通道的检测时序



Gamma

根据下面给出的参数自适应匹配 Gamma 表，**集创提供生成 gamma 表的 dll 库。**

Gamma 表配置需要配合参数

INPUT_BIT_WIDTH ; 设置输入数据位数，范围8 ~ 16

GAMMA_EXPONENT ; Gamma曲率，范围0.0 ~ 5.0

BIT_18 ; 18bit支持设置

GROPU_NUMBER ; 显示分组数，范围1 ~ 127

WIDTH_TOTAL ; 每个分组的 DCLK 个数，范围 0 ~ 255

备注：针对集创给出的 Gamma 表请勿做数据优化。

集创提供生成 gamma 表的 dll 库，不再直接提供 gamma 表，具体使用说明见下节。

DLL 使用说明

集创提供文件包括GammaDll_v2.9.2和IcnGammaTest_v2.9.2

请调用如下函数：

ICNGAMMA_API int GammaV2_Variable_BitWidth(int* GammaOutput, int nInputBitWidth, int nGroupNumber, int nWidthTotal, bool bls18Bit);

Demo Tool使用方法：

1. 进入目录IcnGammaTest\Debug
2. 使用记事本打开icn_gamma.ini，修改gamma参数：
INPUT_BIT_WIDTH=8 ; Range 8 ~ 16,
GAMMA_EXPONENT=0.0; Range 0.0 ~ 5.0, if GAMMA_EXPONENT = 0.0, use Chipone default
gamma curve
BIT_18=0 ; 0 = normal 16bit gamma curve; other = 18bit gamma curve
GROPU_NUMBER=127 ; Range 1 ~ 127
WIDTH_TOTAL=32 ; Range 0 ~ 255
3. 双击IcnGammaTest.exe（或在控制台窗口下执行）
4. 如果正确执行，当前目录下会有gamma.txt产生

DLL使用注意事项：

chipone_gamma.bin和IcnGamma.dll都需要与可执行文件放在同一个目录下。

为方便控制卡厂商移植自动 Gamma 算法，签署 NDA 之后，控制卡厂商可向集创北方索取 IcnGamma.dll 的 source code。

寄存器

上电默认寄存器表

Name	Addr	Reg_R	Reg_G	Reg_B
Reg02	0x02	Bit[5:0]=扫描数-1 Bit[7:6]=00		
Reg03	0x03	Bit[6:0]=刷新率/帧率-1 Bit[7]=0		
Reg04	0x04	0x02	0x02	0x02
Reg05	0x05	0x04	0x04	0x04
Reg06	0x06	0x01	0x01	0x01
Reg07	0x07	0x20	0x20	0x20
Reg12	0x0C	0x1F	0x1E	0x1D
Reg13	0x0D	0x02	0x02	0x02
Reg14	0x0E	0x06	0x06	0x06
Reg15	0x0F	0x01	0x01	0x01
Reg16	0x10	0xdF	0xdF	0xdF
Reg17	0x11	0x21	0x21	0x21
Reg18	0x12	0x01	0x01	0x01
Reg19	0x13	0x00	0x00	0x00
Reg20	0x14	0x00	0x00	0x00
Reg21	0x15	0x00	0x00	0x00
Reg22	0x16	0x00	0x00	0x00
Reg24	0x18	0x15	0x15	0x25
Reg25	0x19	0x00	0x48	0x08
Reg28	0x1C	0Xc0	0xc0	0xc0
Reg29	0x1D	0xA6	0xA6	0xA6
Reg30	0x1E	0x40	0x46	0x48
Reg31	0x1F	0x00	0x20	0x20
Reg32	0x20	0x02	0x09	0x0d
Reg33	0x21	0x01	0x01	0x01

Reg34	0x22	0x1c	0x1c	0x1c
Reg35	0x23	0x00	0x00	0x00
Reg36	0x24	0x01	0x01	0x01
Reg37	0x25	0x02	0x02	0x02
Reg38	0x26	0xAA	0xAA	0xAA
Reg39	0x27	0xAA	0xAA	0xAA
Reg112	0x70	0x00	0x00	0x00
Reg113	0x71	0x00	0x00	0x00
Reg114	0x72	0x00	0x00	0x00
Reg115	0x73	0x00	0x00	0x00
Reg116	0x74	0x00	0x00	0x00
Reg240	0xF0	0x00	0x00	0x00
Reg241	0xF1	0x00	0x00	0x00
Reg242	0xF2	0x00	0x00	0x00
Reg243	0xF3	0x00	0x00	0x00
Reg246	0xF6	0x00	0x00	0x00

声明:

北京集创北方科技股份有限公司保留说明书的更改权，恕不另行通知!

任何半导体产品在特定条件下都有一定的失效或发生故障的可能，用户有责任在使用Chipone产品进行系统设计和整机制造时遵守安全标准并采取安全措施，以避免潜在失败风险及可能造成人身伤害或财产损失情况的发生！

集智创芯，我公司将竭诚为客户提供更优秀的产品！